Notes on Numba's `@njit`: Where to Place the JIT Boundary

1. The JIT Boundary Rule

Numba's `@njit` decorator compiles a Python function to machine code via LLVM. Inside a `@njit`-ed function, every callee must either be another `@njit` function or one of the NumPy / built-in operations Numba understands natively. If a `@njit` function calls plain Python code, Numba either errors out (the default for `@njit`, which implies `nopython=True`) or, with `@jit`, falls back silently to "object mode" and runs at near-Python speed—losing the point of JIT compilation entirely.

The placement rule that follows is then very simple:

(1) The function that contains the *hot loop* (where most of the CPU time is actually spent) must carry `@njit`.

(2) Every helper function called from inside that hot loop must also carry `@njit`.

(3) The outer Python wrapper that orchestrates calls into the JIT'd code does *not* need `@njit`.

Crossing the Python / JIT boundary has a small per-call overhead, of order a few microseconds. As long as each JIT call does substantial work—say, more than a few milliseconds—this overhead is invisible. Conversely, a tight Python loop that calls a fast JIT function many times can be dominated by the boundary cost, and is then the place to also JIT the wrapper itself.

2. Imports

The minimum import is just the decorator:

```
from numba import njit
```

For parallelism, also import `prange` (Numba's parallel range iterator):

```
from numba import njit, prange
```

The common decorator options are:

- `@njit(cache=True)` — write the compiled binary to `__pycache__/*.nbc` so the next interpreter run skips re-compilation. Recommended whenever the function signature is stable.
- `@njit(parallel=True)` — enable Numba's parallel transforms; this is what makes `prange` run on multiple threads.
- `@njit(fastmath=True)` — relax IEEE-754 semantics (allowing reassociation, no signed-zero, etc.) for slightly faster math. Use sparingly: it can change numerical results.

The default `@njit` (no options) is equivalent to `@jit(nopython=True)`, i.e. it refuses to fall back to object mode. That is the safe default—if it errors out, you know something in the function body is not JIT-compatible and you can fix it explicitly instead of silently running slow code.

3. A Minimal Example

Estimating π by Monte Carlo dart-throwing illustrates the pattern in fewer than twenty lines:

```
import numpy as np
from numba import njit

@njit(cache=True)
def sample_point():
    """Sample one (x, y) uniformly in [-1, 1]^2."""
    x = 2.0 * np.random.random() - 1.0
    y = 2.0 * np.random.random() - 1.0
    return x, y

@njit(cache=True)
def count_inside(N, seed=42):
    """Hot loop: count samples falling inside the unit disk."""
    np.random.seed(seed)
    count = 0
    for _ in range(N):
        x, y = sample_point()
        if x * x + y * y < 1.0:
            count += 1
    return count

def estimate_pi(N, seed=42):
    """Plain-Python wrapper: orchestrates and reports."""
    inside = count_inside(N, seed)
    return 4.0 * inside / N

print(estimate_pi(10_000_000))     # ~3.1416
```

Three things to notice:

- Both `sample_point` and `count_inside` are `@njit`-ed. Numba typically inlines the helper into the hot loop, so there is no per-call overhead inside the JIT'd code.
- `estimate_pi` is plain Python. It is called from the user's code (here, the bare `print`), does a single crossing into JIT code, and presents the result.
- The result of crossing the boundary once with $N = 10^7$ samples is that the JIT side does $\sim 10^7$ operations, compared to a few microseconds of boundary cost—a four-orders of-magnitude margin.

4. Compilation Cost and Warm-up

The very first call to an `@njit` function in a Python process triggers LLVM compilation. The cost ranges from about 0.1 s for a trivial helper to several seconds for a function with `parallel=True` or many control-flow branches. Subsequent calls execute the compiled binary directly. With `cache=True`, the *next* Python session loads the cached `.nbc` file from `__pycache__/` instead of recompiling, paying only a few tens of milliseconds.

"Warm-up" in Numba code usually looks like

```
print("Warming up JIT ...")
_ = my_jitted_function(small_problem_size, seed=0)
```

placed immediately before a timed run. Two things are worth being explicit about:

1. **Warm-up does not save total runtime.** Compilation happens once per session in any case. The warm-up call merely shifts when it happens—out of the timed region and into a clearly labelled prelude.
2. **Warm-up is purely cosmetic** with respect to the result. Whether you warm up or not, the output is bit-for-bit identical and the total wall time is the same.

The reason to write a warm-up at all is to keep benchmark numbers honest. Without it, the first timed call reports `compile_time + execution_time`, which can be one to two orders of magnitude larger than the actual computation and gives a misleading "this is slow" impression. With a warm-up immediately before, the timed region reports only `execution_time`.

If you do not measure timings—production scripts that just produce results, CI jobs that diff outputs—the warm-up serves no purpose and can be removed. The actual computation and its total wall time are unchanged.

The real persistent speed-up across Python sessions comes from `cache=True`, not from warm-up. With every JIT-ed function in this repository marked `@njit(cache=True)`, the second and later runs of any script skip compilation entirely and only pay the small `.nbc` load cost. In that situation even the warm-up call itself is essentially free.

5. When to JIT the Outer Wrapper

There are exactly two situations in which the outer wrapper should also be `@njit`-ed:

(1) **The outer loop is itself the hot loop.** If you call a very fast JIT function inside a tight Python `for`, the per-call boundary overhead ($\sim 1\,\mu$s each) can dominate. Putting `@njit` on the wrapper folds the outer loop into the JIT region and removes the crossings.

(2) **You want Numba's parallelism.** A `prange` loop only parallelises when wrapped by `@njit(parallel=True)`. The outer wrapper is then where the parallelism is declared.

The parallel pattern, extending the π example to many independent runs at once:

```
from numba import njit, prange


@njit(cache=True, parallel=True)
def count_inside_many(N_per, N_runs, base_seed=42):
    counts = np.empty(N_runs, dtype=np.int64)
    for i in prange(N_runs):
        counts[i] = count_inside(N_per, base_seed + i)
    return counts
```

Here `count_inside_many` is JIT'd because its loop is what `prange` parallelises, and the inner `count_inside` remains JIT'd because it is called from inside JIT code. The crossings into and out of `count_inside_many` from Python still happen only once.

6. Common Pitfalls

1. **Every callee inside `@njit` must also be `@njit`.** A forgotten decorator on a helper that is called from JIT code is the single most common cause of compile-time errors. Numba's error message points at the offending callee.
2. **No Python objects in hot paths.** Lists, dicts, and arbitrary classes are unsupported (or use Numba's slower typed `List` / `Dict`). Stick to NumPy arrays and scalars.
3. **Avoid allocations in the inner loop.** Returning a tuple of scalars, as `sample_point` does above, is allocation-free. Returning a small NumPy array per call allocates on every iteration and shows up immediately in profiles. The standard idiom is to pass and return scalar components.
4. **`np.random` inside `@njit` uses Numba's own RNG state**, independent of NumPy's global RNG. To make a run reproducible, call `np.random.seed(seed)` from inside the JIT function—not from the Python wrapper.
5. **`cache=True` can serve a stale binary.** If you change the function but it still seems to run the old version, delete `__pycache__/*.nbc` and re-run. Numba normally detects source-level changes, but signature changes or `prange`-related rewrites occasionally slip through the cache check.

7. Applied to This Repository

The `thomson_scattering_numba.py` and `escape_fraction_numba.py` files follow this pattern exactly:

- The packet loop — the function that contains the per-photon `while` — carries `@njit(cache=True)` and is the sole JIT boundary in each file.
- Every helper called from inside that loop (the position and direction samplers, the phase-function sampler, the scattering-rotation routine, the Box–Muller normal sampler) carries `@njit(cache=True)` and exchanges scalar 3-tuples to stay allocation-free.

- The outer routines that sweep over optical depth or albedo values are plain Python. Each call into the JIT'd packet loop does substantial work, so the per-call boundary overhead is negligible.

The parallel variant `thomson_scattering_numba_parallel.py` shows the second pattern from §5: the packet loop is wrapped by `@njit(cache=True, parallel=True)` and uses `prange` to distribute photons across threads.

Ray Traversal on a Cartesian Density Grid

1. Two Algorithms: Baseline & Fast Voxel Traversal

A Monte Carlo radiative transfer (MCRT) calculation on a Cartesian density grid needs, for every free flight and every peeling-off direction, to integrate the optical depth $\tau(\ell) = \int_0^\ell \kappa(x,y,z)\,\mathrm{d}\ell$ along a straight photon path through a piecewise-constant medium. For an optically thick problem this kernel is called many times per packet and is the single largest contributor to the per-photon cost.

This memo compares two algorithms for the same task:

- The **baseline**, used in the Monte Carlo dust-scattering code of [1] (§2.4 and Fig. 3 of that paper). Each iteration recomputes the parametric distances to the next three axis-aligned faces from the current position by three subtractions and three divisions.
- The **fast voxel traversal** of Amanatides & Woo [2], a true incremental Digital Differential Analyzer (DDA) on the voxel grid. Each iteration costs only two floating-point comparisons, one floating-point addition, and one integer increment.

Both yield the *same* sequence of voxels and the same intra-voxel path lengths — they are mathematically identical algorithms. What changes is the arithmetic per step: the DDA amortizes three divisions and three subtractions into the initialization phase, after which the inner loop runs at roughly 3–5× the speed of the baseline (§5.).

2. Grid and Notation

The simulation volume $[X_{\min}, X_{\max}] \times [Y_{\min}, Y_{\max}] \times [Z_{\min}, Z_{\max}]$ is partitioned into $N_x \times N_y \times N_z$ rectangular cells with uniform spacings

$$\Delta X = \frac{X_{\max} - X_{\min}}{N_x}, \qquad \Delta Y = \frac{Y_{\max} - Y_{\min}}{N_y}, \qquad \Delta Z = \frac{Z_{\max} - Z_{\min}}{N_z}.$$

The cell with index (i,j,k), $i \in \{1,\ldots,N_x\}$, $j \in \{1,\ldots,N_y\}$, $k \in \{1,\ldots,N_z\}$, has minimum-coordinate corner $\left(X_i, Y_j, Z_k\right)$ with $X_i = (i-1)\Delta X + X_{\min}$, and maximum-coordinate corner $\left(X_{i+1}, Y_{j+1}, Z_{k+1}\right)$. The opacity per unit length

$$\kappa(i,j,k) \;\equiv\; \sigma\, n\left(\bar{x}, \bar{y}, \bar{z}\right)$$

is constant on each cell. Figure 1(a) illustrates the 2D version of this layout, and Fig. 1(b) shows one step of a ray crossing the nearest axis-aligned face.

The photon is at position $(X_{\mathrm{cur}}, Y_{\mathrm{cur}}, Z_{\mathrm{cur}})$ inside cell (i,j,k) and moving along the unit direction $\mathbf{n} = (n_x, n_y, n_z)$. The job is to find the step length L such that

$$\int_0^L \kappa\left(x(\ell), y(\ell), z(\ell)\right)\,\mathrm{d}\ell \;=\; \tau_{\mathrm{target}}, \qquad (x(\ell), y(\ell), z(\ell)) = (x,y,z) + \ell\,\mathbf{n}, \tag{1}$$

where the right-hand side is sampled from $\tau_{\mathrm{target}} = -\ln\xi$ for a free flight, or from the forced-first-scattering form $\tau_{\mathrm{target}} = \ln[1 - \xi(1 - \mathrm{e}^{-\tau_1})]$ of [3]. Because κ is piecewise constant on the grid, the integral decomposes into

$$\tau(\ell) \;=\; \sum_p \kappa(i_p, j_p, k_p)\,\delta L_p, \tag{2}$$

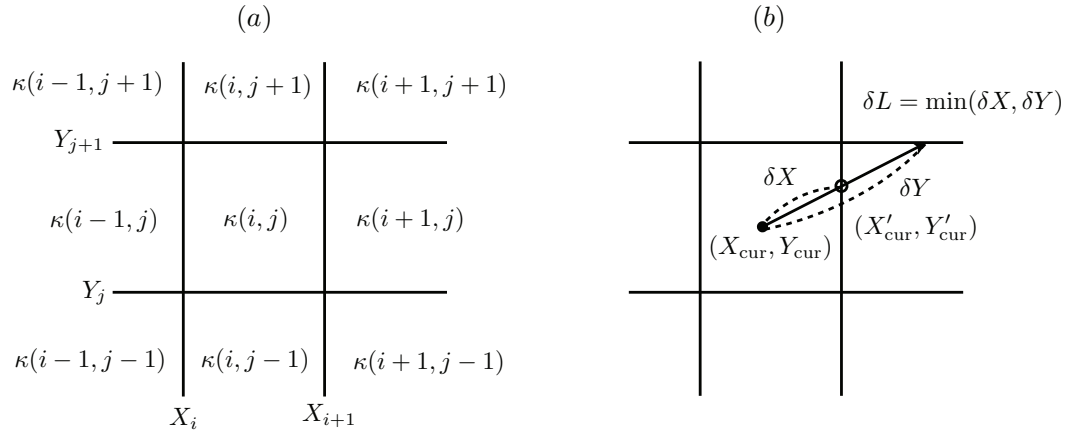

Figure 1: Reproduced from [1], Fig. 3. (a) Two-dimensional sketch of the Cartesian density grid: cell (i, j) stores the opacity per unit length $\kappa(i, j)$, and the faces are the planes $x = X_i, X_{i+1}$ and $y = Y_j, Y_{j+1}$. (b) One step of the traversal: starting from $(X_{\mathrm{cur}}, Y_{\mathrm{cur}})$ along a unit direction (n_x, n_y), the parametric distances to the next x- and y-faces are δX and δY; the actual step is $\delta L = \min(\delta X, \delta Y)$, so the ray exits the current cell through whichever face is reached first.

where δL_p is the path length inside the p-th cell on the ray. Both algorithms in this memo enumerate the cells (i_p, j_p, k_p) and their path lengths δL_p in order; they differ only in how the next δL_p is found.

3. Baseline Algorithm (Seon 2009)

The baseline traversal [1] computes the three parametric distances to the next axis-aligned faces from the current position on every iteration. With the photon at $(X_{\mathrm{cur}}, Y_{\mathrm{cur}}, Z_{\mathrm{cur}})$ inside cell (i, j, k) moving along $\mathbf{n}$,

$$\delta X = \begin{cases} (X_{i+1} - X_{\mathrm{cur}})/n_x, & n_x > 0, \\ (X_i - X_{\mathrm{cur}})/n_x, & n_x < 0, \end{cases} \qquad \delta Y = \begin{cases} (Y_{j+1} - Y_{\mathrm{cur}})/n_y, & n_y > 0, \\ (Y_j - Y_{\mathrm{cur}})/n_y, & n_y < 0, \end{cases} \tag{3}$$

$$\delta Z = \begin{cases} (Z_{k+1} - Z_{\mathrm{cur}})/n_z, & n_z > 0, \\ (Z_k - Z_{\mathrm{cur}})/n_z, & n_z < 0, \end{cases} \qquad \delta L = \min(\delta X, \delta Y, \delta Z). \tag{4}$$

The photon is then advanced by

$$X'_{\mathrm{cur}} = X_{\mathrm{cur}} + n_x\,\delta L, \quad Y'_{\mathrm{cur}} = Y_{\mathrm{cur}} + n_y\,\delta L, \quad Z'_{\mathrm{cur}} = Z_{\mathrm{cur}} + n_z\,\delta L, \tag{5}$$

and the cell index of the next cell is

$$i' = \begin{cases} i+1 & \delta L = \delta X,\ n_x > 0, \\ i-1 & \delta L = \delta X,\ n_x < 0, \\ i & \text{otherwise}, \end{cases} \tag{6}$$

with analogous updates for j' and k'. The optical-depth accumulator is updated by τ_{acc} += $\kappa(i, j, k)\,\delta L$ per step, and a final partial step $\delta L^* = (\tau_{\mathrm{target}} - \tau_{\mathrm{acc}})/\kappa(i, j, k)$ inside the last cell places the scattering point exactly.

Cost per iteration of the baseline. The inner loop performs, on every step:

- Three subtractions and three divisions to compute $\delta X, \delta Y, \delta Z$ [Eqs. (3)–(4)].
- Two floating-point comparisons to take the three-way minimum.
- Three multiplies and three adds to advance the position [Eq. (5)].
- One integer increment to update the cell index.
- One multiply and one add to accumulate τ.

That is, roughly $3\,\mathrm{div} + 3\,\mathrm{sub} + 2\,\mathrm{cmp} + 4\,\mathrm{mul} + 4\,\mathrm{add}$ per step, dominated by the three divisions.

4. Fast Voxel Traversal (Amanatides & Woo)

The Amanatides–Woo algorithm [2] is the *incremental* version of the same traversal. Its key observation is that, along a fixed direction $\mathbf{n}$, the parametric distance the ray must travel between two consecutive x-faces is a constant

$$t_{\delta x} \equiv \frac{\Delta X}{|n_x|},$$

and similarly for y and z. So once the parametric distance to the *first* x-face has been computed, all subsequent x-face distances follow by repeated addition of $t_{\delta x}$. The same is true for y and z. No subtraction or division is ever required inside the loop.

4.1 Parameterization

Write the ray as $\mathbf{p}(t) = (X_{\mathrm{cur}}, Y_{\mathrm{cur}}, Z_{\mathrm{cur}}) + t\,\mathbf{n}$, $t \geq 0$. Because $\mathbf{n}$ is a unit vector, t is the physical arc length along the ray; in particular δL_p of Eq. (2) is just the difference between two consecutive t values.

4.2 Initialization (once per ray)

Given the photon starting cell (i, j, k) and direction $\mathbf{n} = (n_x, n_y, n_z)$, compute and store:

$$\mathrm{step}_x = \begin{cases} +1, & n_x > 0, \\ -1, & n_x < 0, \end{cases} \qquad \mathrm{step}_y, \mathrm{step}_z \text{ analogously}, \tag{7}$$

$$t^x_{\max} = \begin{cases} (X_{i+1} - X_{\mathrm{cur}})/n_x, & n_x > 0, \\ (X_i - X_{\mathrm{cur}})/n_x, & n_x < 0, \end{cases} \qquad t^y_{\max}, t^z_{\max} \text{ analogously}, \tag{8}$$

$$t_{\delta x} = \Delta X / |n_x|, \qquad t_{\delta y}, t_{\delta z} \text{ analogously}. \tag{9}$$

Axis components with $n_x = 0$ (or $n_y = 0$, $n_z = 0$) set the corresponding $t_{\max}$ and t_δ to $+\infty$, so they are never selected. This phase costs three divisions to form the t_δs, three subtractions and three divisions to form the $t_{\max}$s — *the same arithmetic that the baseline does on every iteration*, but done only once.

4.3 Inner loop

Maintain a running t_{cur} (initially 0) and a running τ_{acc} (initially 0). The 3D loop body is just a three-way minimum implemented as two cascaded comparisons:

```
if tMaxX < tMaxY:
    if tMaxX < tMaxZ:
        dL    = tMaxX - t_cur;  t_cur = tMaxX
        tau   = kappa[i, j, k] * dL
        if tau_acc + tau >= tau_target: return final_position()
        tau_acc += tau
        i      += stepX
        tMaxX  += tDeltaX
    else:
        # Z-face crossed: symmetric block
        ...
else:
    if tMaxY < tMaxZ:
        # Y-face crossed
        ...
    else:
        # Z-face crossed
        ...
```

After the comparisons identify the winning axis, the body does: one subtraction ($\delta L = t_{\max} - t_{\text{cur}}$), one assignment ($t_{\text{cur}} \leftarrow t_{\max}$), one multiply and one add (the τ accumulator), one floating-point addition ($t_{\max}$ += t_δ) and one integer increment of the voxel index. The expensive operations that dominated the baseline — three subtractions and three divisions to refresh $\delta X, \delta Y, \delta Z$ — are entirely gone.

4.4 Final partial step

When $\tau_{\text{acc}} + \kappa\,\delta L \geq \tau_{\text{target}}$, the interaction lies inside the current voxel. The remaining sub-cell path length is

$$\delta L^* = \frac{\tau_{\text{target}} - \tau_{\text{acc}}}{\kappa(i,j,k)}, \tag{10}$$

and the final scattering position is

$$x' = X_{\text{cur}} + n_x\,(t_{\text{cur}} + \delta L^*), \quad y' = Y_{\text{cur}} + n_y\,(t_{\text{cur}} + \delta L^*), \quad z' = Z_{\text{cur}} + n_z\,(t_{\text{cur}} + \delta L^*). \tag{11}$$

This last step is identical in cost to the baseline's final step.

5. How Much Faster?

Per-iteration operation counts are summarized in Table 1. The baseline pays three divisions per step; the DDA pays zero, having moved them into the one-time initialization. Divisions are the worst case for FPU throughput (typically

20–40 cycles on modern CPUs vs. a single cycle for an add or compare), so this is where the speedup lives.

Table 1: Floating-point operations per voxel-traversal step. Counts include the τ-accumulation operations $\kappa\,\delta L$ and $\tau_{\rm acc}$ += (one multiply + one add per step), which are the same for both algorithms.

Operation	Baseline (§3.)	DDA (§4.)
Divisions	3	0
Subtractions	3	1
Additions	3	2
Multiplies	4	1
Comparisons (FP)	2	2
Integer increments	1	1
Approx. FLOP / step	~ 13 (incl. 3 div)	~ 6 (no div)

Quantitative estimate. A useful per-step cost model on a current CPU is

$$C \;=\; (\#\,\mathrm{div})\times c_{\rm div} + (\#\,\mathrm{add,\,sub,\,mul,\,cmp})\times c_1,$$

with the heavily-pipelined operations at $c_1 \approx 1\,$cycle and division latency $c_{\rm div} \approx 25\,$cycles (a representative value for double precision `vdivpd`). Then

$$C_{\rm base} \;\approx\; 3\times 25 + 10\times 1 = 85\,\mathrm{cycles}, \qquad C_{\rm DDA} \;\approx\; 0 + 6\times 1 = 6\,\mathrm{cycles}.$$

The per-step speedup is therefore on the order of $\boxed{C_{\rm base}/C_{\rm DDA} \sim 10\text{–}15}$. On hardware where division is fully pipelined or where the compiler hoists the division latency by software pipelining, the gap shrinks to a factor of ~ 3–5; on hardware where division is the bottleneck it is closer to ~ 15. In neither case does the ratio depend on the ray length or the optical depth: it is a fixed multiplier on the cost of every voxel crossing.

Break-even of the initialization. The DDA pays a one-time initialization cost of $6\,\mathrm{div} + 3\,\mathrm{sub} \approx 6\times 25 = 150$ cycles, vs. essentially zero for the baseline. With a per-step saving of $C_{\rm base} - C_{\rm DDA} \approx 80$ cycles, the DDA breaks even after roughly

$$M_{\rm BE} \;\approx\; \frac{150}{80} \;\approx\; 2\ \mathrm{steps}.$$

So a ray that crosses three or more voxels already favors the DDA; in MCRT, the typical photon crosses many cells before its first scattering, so the initialization cost is irrelevant in practice.

Asymptotic cost. Both algorithms visit the same voxels, so each ray crosses the same $M = O(\tau_{\rm sphere}/(\kappa\,\Delta X))$ voxels on average. Total cost per ray is $M\,C + C_{\rm init}$, and the DDA reduces C by an order of magnitude while leaving M unchanged. The overall MCRT wall-clock should accordingly improve by a similar factor in any code whose hot loop is the voxel traversal — which is most pure-scattering codes and the optically-thick limit of absorption-dominated codes.

6. Side-by-Side Pseudo-Code

6.1 Baseline (Seon 2009)

```
def step_baseline(pos, dirn, idx, kappa_grid, X, Y, Z, tau_target):
    x, y, z       = pos
    nx, ny, nz    = dirn
    i, j, k       = idx
    tau_acc       = 0.0

    while True:
        # Recompute the three parametric distances each iteration.
        dX = (X[i+1] - x)/nx if nx > 0 else \
             (X[i]   - x)/nx if nx < 0 else INF
        dY = (Y[j+1] - y)/ny if ny > 0 else \
             (Y[j]   - y)/ny if ny < 0 else INF
        dZ = (Z[k+1] - z)/nz if nz > 0 else \
             (Z[k]   - z)/nz if nz < 0 else INF
        dL  = min(dX, dY, dZ)

        dtau = kappa_grid[i, j, k] * dL
        if tau_acc + dtau >= tau_target:
            dL_star = (tau_target - tau_acc) / kappa_grid[i, j, k]
            return (x + nx*dL_star, y + ny*dL_star, z + nz*dL_star)

        # Advance, snap, update index.
        x, y, z = x + nx*dL, y + ny*dL, z + nz*dL
        if   dL == dX: x = X[i+1] if nx>0 else X[i]; i += sign(nx)
        elif dL == dY: y = Y[j+1] if ny>0 else Y[j]; j += sign(ny)
        else:          z = Z[k+1] if nz>0 else Z[k]; k += sign(nz)
        tau_acc += dtau

        if not (1 <= i <= Nx and 1 <= j <= Ny and 1 <= k <= Nz):
            return None              # photon escaped
```

6.2 Fast Voxel Traversal (Amanatides & Woo DDA)

```
def step_DDA(pos, dirn, idx, kappa_grid, X, Y, Z, tau_target,
             dX_cell, dY_cell, dZ_cell):
    x, y, z       = pos
    nx, ny, nz    = dirn
    i, j, k       = idx
```

```
# ----- Initialization (one division per axis, done once) -----
stepX = +1 if nx > 0 else -1 if nx < 0 else 0
stepY = +1 if ny > 0 else -1 if ny < 0 else 0
stepZ = +1 if nz > 0 else -1 if nz < 0 else 0

tMaxX = ((X[i+1] - x)/nx) if nx > 0 else \
        ((X[i]   - x)/nx) if nx < 0 else INF
tMaxY = ((Y[j+1] - y)/ny) if ny > 0 else \
        ((Y[j]   - y)/ny) if ny < 0 else INF
tMaxZ = ((Z[k+1] - z)/nz) if nz > 0 else \
        ((Z[k]   - z)/nz) if nz < 0 else INF

tDeltaX = dX_cell/abs(nx) if nx != 0 else INF
tDeltaY = dY_cell/abs(ny) if ny != 0 else INF
tDeltaZ = dZ_cell/abs(nz) if nz != 0 else INF

t_cur, tau_acc = 0.0, 0.0

# ----- Inner loop: no divisions, no subtractions of positions -----
while True:
    if tMaxX < tMaxY:
        if tMaxX < tMaxZ:    axis, tHit = 'X', tMaxX
        else:                axis, tHit = 'Z', tMaxZ
    else:
        if tMaxY < tMaxZ:    axis, tHit = 'Y', tMaxY
        else:                axis, tHit = 'Z', tMaxZ

    dL   = tHit - t_cur
    dtau = kappa_grid[i, j, k] * dL
    if tau_acc + dtau >= tau_target:
        dL_star = (tau_target - tau_acc) / kappa_grid[i, j, k]
        tF      = t_cur + dL_star
        return (x + nx*tF, y + ny*tF, z + nz*tF)

    t_cur    = tHit
    tau_acc += dtau

    if   axis == 'X': i += stepX; tMaxX += tDeltaX
    elif axis == 'Y': j += stepY; tMaxY += tDeltaY
    else:             k += stepZ; tMaxZ += tDeltaZ
```

```
    if not (1 <= i <= Nx and 1 <= j <= Ny and 1 <= k <= Nz):
        return None
```

The two codes visit the same voxels in the same order and accumulate the same τ. Only the bookkeeping is different. In the DDA version, the inner loop contains *no* division and *no* position subtraction — only comparisons of three precomputed $t_{\max}$ values and a single addition to refresh whichever $t_{\max}$ was consumed.

7. Practical Notes

- **Snap-to-face is automatic in the DDA.** In the baseline, floating-point round-off can leave the new position just inside or just outside the next cell, and the implementation must explicitly snap the just-crossed coordinate to its exact face value. In the DDA, the cell index is updated by an integer ± 1, and the physical position is only ever reconstructed at the *end* from $t_{\text{cur}} + \delta L^*$. There is no opportunity for the index and the position to disagree.
- **Forced first scattering.** The technique of [3] draws the first τ_{target} uniformly from $[0, \tau_1]$ instead of $[0, \infty)$ and reweights by $w_0 = 1 - \mathrm{e}^{-\tau_1}$. τ_1 is obtained by running either algorithm with an unbounded target until the ray exits the volume and reading off the accumulated τ. The DDA is the better choice here because every ray-to-boundary calculation crosses a large number of voxels.
- **Peeling-off optical depth.** The peeling-off estimator $w_{\text{peel}} = w_n \, \mathrm{e}^{-\tau_{\text{obs}}} \Phi_{\text{HG}}(\Theta) / d_{\text{obs}}^2$ of [4] calls the traversal routine once per scattering event, for every observer direction. This is the single most traversal-heavy phase of a typical MCRT calculation, and is where the DDA speedup pays for itself many times over.
- **Nonuniform axis-aligned grids.** The DDA generalizes immediately to nonuniform spacings if $t_{\delta x}$ is replaced by the local face spacing $(X_{i+1} - X_i)/|n_x|$, recomputed when crossing into a new cell. The inner loop then keeps one division per axis crossing (still better than three per iteration in the baseline).
- **Spherical and cylindrical grids.** The DDA template — "parametric distances to the next boundaries, take the minimum, advance one axis" — generalizes to spherical shells if the $t_{\max}^r$ update is replaced by the smaller positive root of the quadratic $|\mathbf{p} + t\mathbf{n}|^2 = R_{i\pm1}^2$. The arithmetic per shell crossing is heavier (one quadratic solve instead of one add) but the constant-time-per-cell property is preserved.
- **Integer DDA.** Amanatides & Woo's original paper remarks that an integer-only version (a true Bresenham-style line algorithm) is possible if the geometry is normalized to integer spacings. For MCRT, where κ is the heavy operand anyway, the integer optimization is rarely worth the loss of generality.

8. Summary

The Seon 2009 traversal of a Cartesian density grid is a correct voxel-walking algorithm, but it recomputes the parametric distances $\delta X, \delta Y, \delta Z$ on every iteration, costing three divisions and three subtractions per voxel crossing. The Amanatides–Woo Fast Voxel Traversal is the same algorithm with the divisions hoisted into a one-time initialization:

each subsequent voxel crossing costs only two comparisons, one addition, and one integer increment. The two visit the same voxels in the same order; the DDA simply spends much less arithmetic per step.

For typical MCRT use — many scatterings per packet, multiple peeling-off directions per scattering, optically thick clouds with hundreds to thousands of voxels per ray — the per-step speedup of $\sim$ 5–15 translates almost directly into a speedup of the same magnitude in the overall code, because the traversal kernel is the dominant contributor to total cost. The DDA pays for its $\sim$ 150-cycle initialization after roughly two voxel crossings, so the break-even is well below the rays of interest.

References

[1] Seon, K.-I. 2009, "Monte-Carlo simulation of the dust scattering," *Publications of the Korean Astronomical Society*, 24, 43–51.

[2] Amanatides, J. and Woo, A. 1987, "A fast voxel traversal algorithm for ray tracing," *Proceedings of Eurographics '87*, 87, 3–10.

[3] Witt, A. N. 1977, "Multiple scattering in reflection nebulae. I. A Monte Carlo approach," *The Astrophysical Journal Supplement Series*, 35, 1.

[4] Yusef-Zadeh, F., Morris, M., and White, R. L. 1984, "Bipolar reflection nebulae: Monte Carlo simulations," *The Astrophysical Journal*, 278, 186.